

Module 6 — Microservice Architecture Report

Grad Café Analytics: Docker, RabbitMQ & Containerized Services

1. Docker Compose Architecture

The application is decomposed into four services defined in **docker-compose.yml**, each with a specific role, health check, and dependency ordering:

Service	Image/Build	Port	Role
db	postgres:16	5433:5432	PostgreSQL database; schema auto-initialized via db/init.sql
rabbitmq	rabbitmq:3.13-management	15672:15672	Message broker; management UI at localhost:15672
web	./src/web (build)	8080:8080	Flask app; publishes tasks to RabbitMQ, serves dashboard
worker	./src/worker (build)	—	Consumes tasks from RabbitMQ; scrapes, cleans, inserts data

All services share a private Docker network (Compose default). PostgreSQL data is persisted in a named volume (*pgdata*). The *web* and *worker* services depend on both *db* and *rabbitmq* being healthy before starting, enforced via **condition: service_healthy** in the `depends_on` block. The worker mounts *src/data/* read-only so it can access the seed JSON without baking it into the image.

To start the full stack:

```
docker compose up --build
```

To seed the database from the host machine:

```
export DATABASE_URL="postgresql://postgres:<password>@localhost:5433/gradcafe"
python src/db/load_data.py
```

2. RabbitMQ Message Flow

All long-running and data-modifying work is decoupled from the web tier using RabbitMQ. This ensures the Flask app remains fast and stateless — button clicks return immediately with HTTP 202 while the worker handles the actual processing asynchronously.

Message flow:

1. User clicks **Pull Data** or **Update Analysis** in the browser.
2. Flask calls **publish_task(kind, payload)** in *publisher.py*.
3. A compact JSON message is published to the durable direct exchange **tasks** with routing key **tasks**, `delivery_mode=2` (persistent).
4. Flask returns **HTTP 202** immediately — `{"status": "queued"}`.

5. The worker's **consumer.py** receives the message from queue **tasks_q**.
6. The worker opens a DB transaction, routes by **kind**, executes the handler.
7. On success: commits the transaction and calls **basic_ack**.
8. On error: rolls back and calls **basic_nack(requeue=False)** — no infinite retries.

AMQP entities (declared idempotently on both web and worker):

Entity	Name	Type	Durable
Exchange	tasks	direct	Yes
Queue	tasks_q	—	Yes
Binding	tasks → tasks_q	routing key: tasks	—

Backpressure: The worker sets **prefetch_count=1** via **basic_qos**, ensuring it processes one message at a time. This prevents the worker from being overwhelmed during high message volume and gives each task the full DB connection it needs.

3. Worker Behavior & Task Handlers

The worker opens a fresh PostgreSQL connection per message. Each handler runs inside a transaction — commit only happens on success.

scrape_new_data handler:

1. Reads the **last_seen** watermark from *ingestion_watermarks* table.
2. Calls **scrape_data()** to fetch new records from GradCafe.
3. Cleans records via **clean_data()**.
4. Inserts with **ON CONFLICT (url) DO NOTHING** for idempotency.
5. Advances the watermark to the max `date_added` seen in this batch.
6. Commits and acks.

recompute_analytics handler:

1. Refreshes any materialized views (if present) within the transaction.
2. Calls **get_all_results()** to warm the query cache.
3. Commits and acks.

Watermark table schema:

```
CREATE TABLE ingestion_watermarks (  
  source      TEXT PRIMARY KEY,  
  last_seen   TEXT,  
  updated_at  TIMESTAMPTZ DEFAULT now()  
);
```

4. Database Initialization

The PostgreSQL schema is initialized automatically when the stack first starts. The file **db/init.sql** is mounted into the container at `/docker-entrypoint-initdb.d/00-init.sql`, which PostgreSQL executes on first startup. This creates both the *applicants* and *ingestion_watermarks* tables with all required columns and constraints. SQL queries continue to use parameterized psycopg2 composition from Module 5 — no raw string interpolation.

5. Build, Run & Verification Steps

After running **docker compose up --build** and seeding the database:

1. Visit **http://localhost:8080** — Flask dashboard loads with real data.
2. Click **Pull Data** — green banner appears: 'Data pull queued! Worker is processing.'
3. Click **Update Analysis** — green banner: 'Analytics recompute queued!'
4. Visit **http://localhost:15672** (guest/guest) — RabbitMQ dashboard shows 1 connection, 1 queue, 1 consumer.

6. Docker Hub Image Publishing

Both service images are tagged and pushed to Docker Hub under the public repository **scharfshutzer/module_6**:

```
docker tag module_6-web scharfshutzer/module_6:web
docker tag module_6-worker scharfshutzer/module_6:worker
docker push scharfshutzer/module_6:web
docker push scharfshutzer/module_6:worker
```

Pull instructions: **docker pull scharfshutzer/module_6:web** or **docker pull scharfshutzer/module_6:worker**

Registry: https://hub.docker.com/r/scharfshutzer/module_6

7. Known Issues & Deviations from Ideal

The following deviations were made intentionally due to architectural or environmental constraints. Each is documented here for transparency:

Coverage omissions (worker/etl files)

`src/worker/etl/clean.py`, `db_config.py`, and `query_data.py` are identical copies of their counterparts in `src/web/app/`. They exist because Docker containers have isolated filesystems — COPY . . only copies

files within each service directory. These files are covered via the web/app versions. They are omitted in .coveragerc with a documented explanation in the README.

src/web/app/db_config.py omitted from coverage

Tests use src/db/db_config.py directly. The web/app version is identical and omitted from coverage for the same Docker isolation reason described above.

.pylintrc similarity threshold

The [SIMILARITIES] section sets min-similarity-lines=10 to suppress the duplicate-code warning for the 5-line RabbitMQ channel setup that appears in both publisher.py and consumer.py. Both files genuinely need this boilerplate — moving it to a shared module is not possible without restructuring the Docker build contexts.

consumer.py main() marked pragma: no cover

The main() function starts a blocking RabbitMQ connection loop that cannot run in a unit test environment without a live broker. It is marked # pragma: no cover. All other consumer functions including handlers, _on_message, and _open_db are fully tested.

DB port mapped to 5433 externally

docker-compose.yml maps db:5432 to localhost:5433 on the host to avoid conflict with a locally running PostgreSQL instance on port 5432. Inside Docker, all services communicate using db:5432 normally. load_data.py must be run with localhost:5433 when seeding from the host machine.

Manual database seeding required

The applicant data is not automatically seeded on docker compose up. The professor must run: export DATABASE_URL=postgresql://postgres:@localhost:5433/gradcafe followed by python src/db/load_data.py to populate the database. The schema is created automatically via db/init.sql on first startup.